Modeling context associations for Semantic Web ecosystems

Abstract. The inclusion of personal data in Web ecosystems—such as the Solid Project and thematic data spaces—requires the integration of context information in the form of provenance, policies, and signatures over exchanged data. This enables these ecosystems to fulfill requirements for managing data access, stating usage restrictions, and assigning confidence to data. Currently, the integration of context is driven by protocol development that limits both the expressiveness of context and its reuse outside ecosystem boundaries. This work explores an alternative approach based on the unified modeling of data and context in RDF. This enables the use of RDF tooling such as SPARQL for the evaluation of combined requirements for data and context, while ensuring existing RDF infrastructure can be used for the exchange, storage, and integration of such context associations in Web ecosystems. To support this, we propose a set of modeling restrictions for context associations that ensure consistency across RDF-based ecosystems. Based on these restrictions, we provide a set of candidate models that are adapted from RDF annotation models. These are evaluated for their effectiveness through compliance with the stated requirements, as well as regarding the scaling behavior of query evaluation when expressing combined requirements for context and data over these associations. Our findings show that both triple collections and blank node graphs effectively support the modeling and evaluation of context associations for use in Web ecosystems and can scale linearly with dataset size for combined queries for data and context. We conclude that RDF can model combined expressions of data and context for use in Web ecosystems today. Existing context models can largely be reused in the local scope to define multiple context definitions in a single RDF exchange, improving the ecosystem's capabilities in expressing and integrating context over data, and improving context reuse outside ecosystem boundaries.

1. Introduction

In a 2003 presentation [1], Sir Tim Berners-Lee put forward a set of challenges for the Semantic Web. In these challenges, the concept of a "Web of Trust" is mentioned, indicating the need for actors to trust the data they integrate, and the accompanying requirement for availability of context information such as provenance and signatures in Web ecosystems. The need for context information is reflected in modern architectures for Web ecosystems. Initiatives such as the Solid project [2] and thematic data spaces[1] address use cases involving both personal and business data management, introducing additional requirements for exchanging verifiable claims for resource ac-

cess management [2], expressing usage restrictions over exchanged data [3], and tracking provenance[2] within ecosystems [3]. The integration of context information is also addressed in regulatory frameworks. Article 15[4] of the General Data Protection Regulation (GDPR) grants data subjects the right to access their personal data along with information about its provenance and processing. Additionally, Article 20[5] establishes that data subjects have the right to data portability, enabling them to transfer their personal data between controllers, where the availability of context information enhances the receiving controller's ability to integrate the transferred data.

These initiatives for data spaces, digital identity, digital wallets, and data protection regulations show a digital landscape increasingly geared towards enhancing the data-sharing capabilities of online ecosystems. Integration of context in these ecosystems is mainly implemented via ecosystem protocols that provide functionality such as identity management, enforcement of access control and the exchange of verifiable claims, enforcement of usage restrictions over data exchanges, and management of provenance. Where such protocols provide integration of context within the ecosystem boundary, the reuse of available context information outside these ecosystems is limited due to differences in the protocols used, varying modeling and formatting of claims, or reliance on implicit knowledge in the ecosystem such as a certification process for user registration in the ecosystem.

With this work, we contribute to the ongoing discourse by evaluating the feasibility of expressing context over data directly in the RDF data model. Decoupling the expression of context information from concrete protocols into structured requests for data and associated context in RDF increases the flexibility both in requesting and exchanging specific context requirements, as well as improving reuse by explicitly defining context over exchanged data. The restriction to RDF enables these expressions to be exchanged, stored, and integrated across ecosystems using existing RDF tooling. This way, integration of context directly in the data exchanges of ecosystems decreases reliance on specific protocols for the context management.

To illustrate the point, we present a simple exchange involving Bob, a user with a Web identity and a personal data space under his control. When Bob visits an online store, the store detects his data space and requests verifiable information about his name and age to personalize the experience and show age-restricted content, as shown in Listing 1.

```
# Request
CONSTRUCT
    { <http://people.org/Bob> foaf:name ?name; foaf:age ?age. }
WHERE
    { <http://people.org/Bob> foaf:name ?name; foaf:age ?age. }

# Raw data
<http://people.org/Bob> foaf:name "Bob".
<http://people.org/Bob> foaf:age 26.
```

Listing 1: Querying raw data requires protocol support to carry the context requirements.

Where this satisfies the requirement for raw data, the store needs to rely on ecosystem support both to verify the authenticity of the incoming information, and to ensure that the shared data can be used for the purpose of providing its service. The necessary usage agreements and signatures are handled by the ecosystem protocols, and exchanged separately from the data as shown in Fig. 1.

Contrasting this, the inclusion of context with data directly in the message exchanges of ecosystems enables direct queries for data with context requirements. Now, the updated store request includes: (i) the user name and age, (ii) a usage agreement granting permission to use *(i)* for the purpose of service provision, (iii) a signature over *(i)* from the government registry to ensure data authenticity, and (iv) a user signature over *(ii)* to verify the authenticity of the policy. Structuring this request as a SPARQL Construct query, Bob's data store can now provide both the required data and context in a single information exchange, as shown in Fig. 1.

Integrating both the data and context, this data can now be reused for new use cases while ensuring adherence to usage agreements, to prove processing of data for auditing purposes or to comply with legal requests for portability of data and associated context.

The rest of this paper is structured as follows: Section 2 provides an overview of related work. In Section 3, we contribute a set of base requirements to which modeled context associations have to adhere. With Section 4 we provide a set of candidate models by adapting prominent Metadata Representation Models to the stated requirements. In Section 5 we evaluate the approach based on a theoretical analysis of industry requirements, as well the scaling behavior of the proposed models, after which we conclude and discuss future work in Section 6.

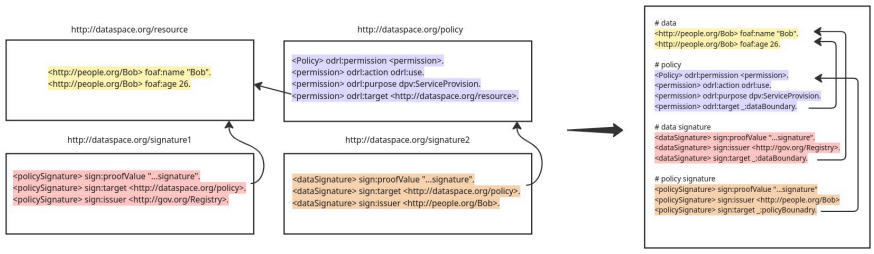

Fig. 1: Modeling associated context within the RDF scope instead of as separate resources enables combined exchange, storage and integration of context and data.

2. Related Work

The Resource Description Framework (RDF) inherently supports the expression of context over Web resources. As the name suggests, it describes resources by making statements that reference the resource URI to state information about that resource in the form of relations to other resources or literal values. Publishing RDF data as a Web resource, the URI of this data resource can be referenced by an expression of context to indicate provenance or policy information attached to the data resource.

Where such expressions of provenance over Web resources can be integrated in ecosystem protocols, the combined evaluation of requirements for context and data in these cases is not supported by native RDF tooling such as SPARQL.

To address this shortcoming, a viable approach here is to extend SPARQL with functionality to keep track of context during the query evaluation process. Examples here include tracking triple provenance during query evaluation in [4] and the filtering of results in query evaluation based on trust values assigned to individual triples with tSPARQL [5]. This integration of context information in the evaluation process enables its inclusion in Web ecosystems, but is restricted in its functionality in supporting only specific expressions of context such as provenance or concrete trust values assigned to triples. This contrasts with our goal of capturing generic expressions of context in RDF that can be queried with SPARQL while retaining the flexible and expressive nature of core RDF.

To annotate statements or sets of statements in RDF with metadata, general annotation approaches such as reification, rdf-star [6], triple coloring [7] and more have been proposed and seen inclusion in the core RDF specification. These enable generic expressions of metadata over statements in RDF, with named graphs as proposed in [8] stating the need for the expression of policies, signatures and restrictions for access

and usage as driving factors in their use. We build on these annotation models in Section 4 to investigate their usability in a Web setting for interoperable data exchange.

Expressions of provenance in RDF have seen wide adoption through strong core vocabularies such as Dublin Core (DC)[6] and the Provenance Ontology (PROV-O)[7]. Policy expression has gravitated towards the use of the Open Digital Rights language (ODRL) [11] for the development of Web ecosystems [12] [8] [9]. These models for the expression of provenance and policies reference the target of their expression as URIs. Such a resource target implies that for example the policy is expressed over both the resource and its contents. However, as no formal interpretation is given in these models for how the target should be interpreted, the provision of a local identifier that is interpreted as denoting a set of statements in the local scope can be used to define local expressions of provenance and policies over data without needing to change the expression or interpretation of the context models.

In the case of ODRL, the W3C community group in which it is developed[10] has published a use cases document[11] stating a need for (i) the licensing of content and embedded artefacts based on conditions for usage and attribution and (ii) indications of data quality through policies for data endpoints. For both these cases, the local in-place expression of policy information over data in RDF resources proves beneficial. The ability to target specific subsets allows for multiple expressions of policy in the same document, which can be used to target both a source artefact such as a blog post, while including separate policies over embedded artefacts such as embedded images or added comments that retain the author policy. Similarly, it enables indications of data quality at a smaller granularity within resources.

Similar to the use of the ODRL model for policy expression, signature definitions have largely converged to the use of Verifiable Credentials (VC)[12]. The ability to receive signed information is an underlying requirement for many systems in Web ecosystems, such as verifying claims for access management, verifying authenticity of both data and context. The VC model requires expression as JSON-LD, both for compatibility with JSON tooling, and to indicate the concrete boundary of the `credentialSubject` through its JSON structure. This reliance on a JSON structure makes its adoption in RDF systems difficult, indication of which can be found in the use of JSONPath to state policy constraints over VCs in ODRL policies[13].

Alternative to this JSON interpretation, RDF Dataset Canonicalization[14] enables the creation of hashes over sets of quad statements in RDF. This enables the creation of signatures over referenceable sets of statements in RDF, by creating a hash value of

the statement set over which a signature is created. This resulting signature can then again be modeled in RDF and reference the signed set of statements as the target of its signature.

3. Defining modeling requirements for context associations

The integration of context in data exchanges for Web ecosystems requires the guarantee that the interpretation of both the data and expressed context over the data remains consistent throughout the ecosystem in which it is used. The modeling of such associations of context information to target sets of statements in RDF, or 'context associations' in short, requires that context statements can reference an identifier that denotes an exact set of target statements. The exchange, storage and integration of context associations requires them to be defined in the local RDF scope. This can be the exchange of RDF data in a network, the document or graph store in which they are contained, or the scope of the processing step in which they are integrated. We now define three requirements that formalize these requirements, ensuring that modeled context associations that adhere to them can be trusted to correctly present both context and data throughout the ecosystem in which they are used.

The first requirement in the modeling of a context association is that an exact set of statements can be defined in the local RDF scope of its document, store or processing step, that can be referenced from this local scope. The definition of an exact set of statements in RDF is a core requirement for the consistency of their interpretation, with hashes of their canonicalized representation[14] supporting completeness guarantees.

(R1) A context association must define an exact set of statements in the local scope that can be referenced from this local scope.

Secondly, this set of statements defined in the local scope must be immutable when combined with externally defined data. This requirement is important, as in RDF, the combination of data from multiple sources in the exchange, storage or processing of context associations can lead to unwanted merges causing incorrect interpretations. To prevent such behavior their modeling in RDF must prevent such merges from interfering with their expression of context over target sets of statements.

(R2) A context association must remain immutable under exchange, storage and processing with external data.

Third, it is important that the modeling approach can be recursively defined. As stated in Section 1, use cases include tracking provenance in ecosystems and expressing signatures over policy and provenance information defined in the local scope. The inability to define recursive expressions of context would require external systems to bridge this shortcoming, decreasing their ability to be processed in their combined expression by tooling such as SPARQL.

(R3) A context association model must not prevent recursive expressions of context.
Conforming to the above requirements ensures that context associations have a consistent interpretation throughout the Web ecosystems in which they are used. This ensures that signatures, policies and provenance expressed over sets of statements remain consistent in interpretation when combined with similar expressions of context from other sources.

4. Modeling referenceable sets of statements in RDF

This section presents a selection of candidate models for the definition of context associations in RDF. For this, we base ourselves on annotation mechanisms in RDF [13], that are adapted according to the requirements stated in Section 3.
Consistent in all proposed approaches is the use of blank nodes to define the referenceable sets of statements in the local scope. External data sources can accidentally or intentionally reuse URI identifiers used in the local scope. In contrast, blank node identifiers are intended to be defined and used in the local scope. While not explicitly enforced, the RDF specification indicates that when combining data from multiple sources, reused blank node identifiers from different sources should be relabeled or 'standardized' by replacing them with non-clashing identifiers. As the consistent relabeling of all blank nodes in a source to a different blank node identifier does not change its meaning, we take the automated relabeling of blank nodes in external data as good practice when integrating RDF. This reliance on blank node identifiers is the only RDF-native approach to prevent clashing of externally defined identifiers, enabling us to comply with the immutability requirement as stated by *(R2)* in Section 3.
In cases where external references are required, skolem identifiers can be used [14]. Skolem identifiers ensure the unique generation of identifiers at the time of their creation. Where this provides a uniqueness guarantee, it does not however prevent external sources from reusing the locally defined skolem identifiers, and hence does not provide the same guarantees in preventing identifier clashes when merging with external data sources.

4.1. Graph-based annotation

The use of named graphs to model RDF graphs within the local scope of a single document was popularized in [8]. This work, dating back to 2005, proposed named graphs as a solution for expressing provenance chains, access controls, usage restrictions, signatures and other forms of contextual information over RDF graphs. The paper mentions two problems in how different sources using named graphs with the same graph names should be handled, and how named graphs should be interpreted. The authors state that different graphs with an identical name are contradictory and

that the interpretation of named graphs depends on the external definition of a set of *accepted* graphs, for which the result is interpreted as the union of these accepted graphs.

Named graphs (ngraph) saw their inclusion in RDF with version 1.1 of the specification. They are defined in the RDF syntax as a pair of an RDF Graph and a graph name. Where the syntax is well-defined, the semantic interpretation of the RDF Graph of a Named Graph, and the relation between its graph name and the RDF Graph are intentionally left open to interpretation. By not specifying a concrete interpretation for the graph name in reference to the connected RDF Graph of the pair, their use in interoperable data exchanges in Web ecosystems is limited to managed ecosystems. This lack of referenceability is in violation of the requirement for defining exact sets of statements that are referenceable from the local scope *(R1)*.

Blank node graphs (bngraph) are a proposed extension to named graphs that provide a fixed interpretation, presented in [15]. Blank node graphs are defined as named graphs in RDF with a blank node term as the graph name, for which a concrete interpretation is enforced in the graph name being a reference to the connected RDF graph. This is enforced through the use of a content type profile [16] or by explicitly typing the graph name term as a graph identifier, as defined in the Context Association specification document[15].

```
# RDF Resource
@prefix ca: <http://w3id.org/context-associations/>.
_:data { ... }
_:data a ca:GraphIdentifier.
...

# Query
PREFIX ca: <http://w3id.org/context-associations/>
CONSTRUCT { ... } WHERE {
    ?dataGraph a ca:GraphIdentifier.
    GRAPH ?dataGraph { ... }
    ...
}
```

Listing 9: Modeling context associations using *bngraph* results in human readable queries for data and context using the GRAPH function.

By enforcing an interpretation for the graph name as a reference to the connected RDF graph, blank node graphs model an exact graph of statements that can be referenced in the local scope *(R1)*. Where the RDF specification dictates that named graphs with the same name combined from different sources must be merged on com-

bining the sources, the use of blank nodes for the graph name prevents this merging of named graphs *(R2)*. Recursive modeling of context associations is similarly straightforward. By modeling the context information in a context graph as shown in Listing 3, this context graph can then be targeted by additional context *(R3)*, enabling, for example, the signing of policies expressed over data. The querying of the resulting associations in SPARQL can make use of the `GRAPH` function to indicate that statements are contained in a named graph. A SPARQL Construct query can now assert concrete statements based on combined requirements for data and context that extracted from the available blank node graphs. A downside to this is that the SPARQL specification does not include the ability to create named graphs as the result of a Construct query, preventing the generation of new context associations using SPARQL Construct. This is not an architectural shortcoming, as existing implementations of SPARQL engines supporting the creation of Named Graph in construct queries are in use in Apache Jena[16].

4.2. Triple-based annotation

Next to named graphs, annotation of triples in RDF is also common practice. Defining statements about individual triples however has limited practical application in Web ecosystems where context needs to be defined over larger selections of triples. Where this is possible expression of provenance and policies to target individual triples, this requires context models to support atomization, in that their interpretation when expressed over the individual triples must be equivalent, which is not the case for signatures. Additionally, this would incur a total cost of `(x*n) + (x*m) * n` statements to express a set of `m` context statements about a set of `n` data statements when modeling recursive context, leading to unacceptable performance.
However, triple annotation enables the modeling of collections of triples, such as a retrieval entity that contains all the triples in the retrieved document. We generalize this approach in defining an entity of type `ca:Collection` as a collection of triples. Using this collection entity, we evaluate the standard reification (stdreif), n-ary relations (naryrel), singleton propertt (sgprop) and triple term (tterm) approaches in the modeling of collections of statements over which context information is defined.

```
@prefix ca: <http://w3id.org/context-associations/>.

_:Collection a ca:Collection;
    ca:contains _:statement1, statement2, ... .
```

Listing 4: Triple annotation requires the modeling of a collection entity in which a set of triples is contained, that can be targeted by context information.

Reification (stdreif) A first approach to annotating statements in RDF is reification. Its inclusion in the RDF vocabulary and specification enables the deconstructing of triple statements into separate statements defining its `subject`, `predicate` and `object` terms on a new reifier subject. This reifier subject can now be targeted by the collection entity to indicate the reified triple is contained in the collection, as shown in Listing 11.

```
_:dataCollection a ca:Collection;
    ca:contains _:b0, ...

_:b0 a rdf:Statement;
   rdf:subject <http://people.org/Bob>;
   rdf:predicate foaf:name;
   rdf:object "Bob";
```

Listing 11: *stdreif* approach to model collections of statements.

stdreif provides a straightforward modeling of contained statements. The reified triples contained in the collection entity are not considered asserted. Using SPARQL Construct, asserted triples can be extracted from such a collection.

N-ary relation (naryrel) is a modeling approach that makes use of a relation object that can be annotated with metadata. Where this is a useful guideline in the development of data models where contextual information needs to be included on relations, the approach can be generalized through a concrete conversion of the relation between terms into a relation object that can be annotated, as shown in Listing 12.

```
_:dataCollection a ca:Collection;
    ca:contains _:d1, ...

<http://people.org/Bob> foaf:name _:d1.
_:d1 foaf:name_value "Bob".
```

Listing 12: *naryrel* approach to model collections of statements.

Where theoretically viable, this approach requires the addition of concrete conversion rules, without providing additional value compared to the *stdreif* approach.

Singleton property (sgprop) is a modeling approach that defines a unique property for a triple that can be annotated with metadata, as seen in Listing 13. The relation from which the singleton property is derived is linked as `rdf:singletonPropertyOf`.

```
_:dataCollection a ca:Collection;
    ca:contains foaf:name#1, ...

<http://people.org/Bob> foaf:name#1 "Bob";
foaf:name#1 rdf:singletonPropertyOf foaf:name.
```

Listing 13: *sgprop* approach to model collections of statements.

The use of the predicate position to define the singleton property prevents the use of blank nodes, as these cannot be defined in the predicate position. Skolem identifiers can be used, but these do not prevent interference by externally defined statements when merging.

Triple term (tterm) is a proposed addition to the RDF specification for version `1.2`. Building on the RDF-star proposal [6], triple terms allow the definition of a triple as a term in the RDF domain model and syntax, enabling them to be defined as the object of another triple. This direct embedding of triples in other triples allows both compact modeling of collections and the queries evaluated over them.

```
_:dataCollection a ca:Collection;
    ca:contains <<<http://people.org/Bob> foaf:name "Bob">>, ...
```

Listing 14: *tterm* approach to model collections of statements.

Compared to the *stdreif* approach, triple terms has the advantage of being compact in nature both in the modeling of triple collections and SPARQL queries, improving human readability. Their novelty however has the downside of implementations in existing graph stores being either unavailable or experimental.

5. Evaluation

To evaluate the ability to model the combined expression of data and associated context in RDF, in Subsection 5.1 we provide an evaluation of the modeling approaches presented in Section 4 in their compliance with the modeling requirements proposed in Section 3. To evaluate the usability of these modeling approaches in real world scenarios, we provide an analysis of how query evaluation scales proportional to dataset size when evaluated over context associations stored in local graph stores. For this, we use the example presented in Section 1 as a concrete scenario that requires the assertion of data on combined requirements for data and recursive expressions of context.

5.1. Evaluation of modeling requirements

In Section 3, we presented three requirements to which context association models should comply to ensure the resulting associations are viable for use in the exchange, storage and integration of combined context and data across Web ecosystems. These are: *(R1)* a context association must define an exact set of statements in the local scope that can be referenced from this local scope, *(R2)* a context association must remain immutable under exchange, storage and processing with external data, and *(R3)* a context association model must not prevent recursive expressions of context.
Evaluating the modeling approaches proposed in Section 4 according tho these requirements, we observe the following:

Approach	type	R1	R2	R3
ngraph	named graph	**No**	Yes	Yes
bngraph	named graph	Yes	Yes	Yes
stdreif	collection	Yes	Yes	Yes
naryrel	collection	Yes	Yes	Yes
sgprop	collection	Yes	**No**	Yes
tterm	collection	Yes	Yes	Yes

The *ngraph* approach does not fulfill the criteria *(R1)*, as in RDF, the interpretation of the graph name in reference to the RDF graph of a named graph pair is not specified. This lack of specification renders the named graphs unusable to reference sets of statements.
The *bngraph* approach resolves this shortcoming by defining a concrete interpretation for the blank node used as the graph name in reference to the connected RDF Graph under the name of blank node graphs. Where this interpretation is not native to the RDF specification and is syntactically equivalent to named graphs, the use of a content type profile and the explicit typing of the graph name in combination with the use of a local blank node identifier limits potential for misinterpretation. The assertion of statements from the blank node graphs is achieved with SPARQL Construct.
The *stdreif* approach provides an effective way of modeling context associations in RDF, represented as statements referencing a collection entity containing the target statements in their reified form. It has the advantage of restricting the data model to RDF 1.0, enabling the use of legacy RDF systems to store, exchange and integrate these expressions of context. Requiring four statements to represent a single triple in a collection, this approach incurs the largest cost in statement count in the modeling of collections of triple statements. Where the reified statements do not assert the original statement, these can be asserted as the result of a SPARQL Construct query.

The *naryrel* approach is capable of compliance with the requirements for creating context associations. It comes with additional overhead compared to the other approaches in requiring an agreement on how these relation objects should be defined across ecosystems to prevent misinterpretation.

The *sgprop* approach fails to ensure immutability. Its use of dynamic predicate values to identify individual statements prevents the use of blank nodes for these predicates, as this is not supported in RDF. Skolem identifiers can be used to mitigate this issue, but this does not ensure immutability from external sources.

The *tterm* model behaves similarly to the *stdreif* model, but can directly define the contained triples as terms in the collection model. Their use reduces the amount of statements needed to represent a contained triple, and provides a concrete semantic interpretation of the triple terms contained in a collection entity in the RDF domain model. Their novelty however has the downside of implementations in existing graph stores being either unavailable or experimental.

From this analysis, we conclude that both graph-based models and triple-based models in the form of triple collections can model the target sets of triples needed for the modeling of context associations in the local RDF scope. Comparing the above modeling approaches, we find the *bngraph*, *stdreif* and *tterm* approaches are preferable in their reliance on standardized syntax expressions in RDF, where *naryrel* imposes additional reliance on for converting triple statements. From a developer standpoint, the use of named graphs in *bngraph* provides the best readability for the expression of context associations and the queries used to evaluate them.

5.2. Evaluation of query evaluation scaling behavior

To evaluate the models in a real-world setting, we include an analysis of the scaling behavior for the evaluation of SPARQL queries over a dataset consisting of user data and associated context. We extend the example presented in Section 1 of a user store collecting user data when negotiating access, by having the store query information retrieved in its user interactions for reuse in another use-case. For this use-case, the store wants to reuse the user age data for the purpose of marketing. However, to ensure that this is compliant with the conditions under which data was exchanged, this request is modeled as a query for user age, for which an associated usage agreement is available stating allowing the intended processing purpose, and the availability of signature information to ensure both the authenticity of the age information and the usage agreement.

The evaluation of the scaling behavior of the proposed models under query evaluation was run with a Dell Latitude 5540 laptop with a core i7-1365U CPU and 16GB of RAM memory. Specific datasets of different sizes were generated for the purpose of this evaluation using both the *bngraph* and the *stdreif* approach. As triple terms are

expected to be specified with the release of RDF 1.2, current graph stores do not yet provide an implementation. We experimented with falling back on RDF-star, for which experimental support is available in most graph stores. However, as we saw noticeably worse scaling compared to the reification approach in our testing, we decided to not include these results. Datasets were generated both using blank nodes and skolem identifiers for the graph name and collection identifiers to see if this influences execution due to different indexing strategies. The codebase containing the dataset generation, the evaluated queries and the benchmark execution is made available online[17].

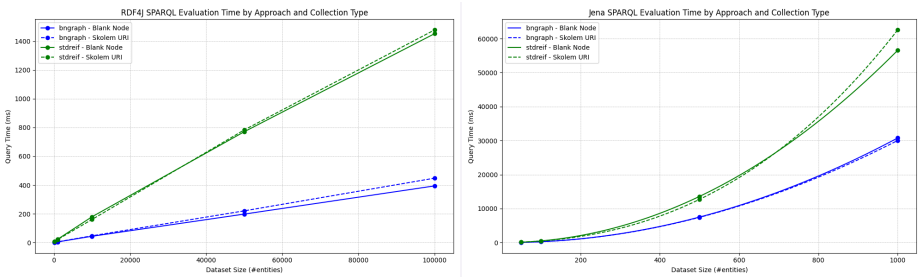

Fig. 2: Note the differing axes. The difference in linear scaling over the RDF4J implementation compared to the slower and non-linear scaling Apache Jena implementation indicates that query evaluation can scale linearly with dataset size, but is heavily dependent on the indexing approach of the implementation to achieve linear scaling.

From the evaluation of scaling behavior, the results of which can be found in Fig. 2, we conclude that linear scaling of query evaluation is possible for context associations modeled in RDF. The RDF4J benchmark results show linear scaling in query evaluation time to dataset size for both graph-based and collection-based modeling. However, the Apache Jena benchmark indicates that this scaling behavior is dependent on the graph store and SPARQL query engine implementation. We expect that the worse performance of the Jena implementation can be explained through differences in indexing and query planning. Additionally, we see that the Blank Node Graph approach based on the use of named graphs performs noticeably better than the collection based approach in both instances. This result is expected, as most graph stores include named graphs in their indexing, where for collection models the relation between an identifier in a triple and the identifier of the collection in which it is contained is indirect, causing decreased performance. No meaningful result was observed in the use of skolem identifiers compared to blank node identifiers for both the graph name and the collection identifiers.

6. Conclusion

The work in this paper demonstrates that RDF provides the necessary modeling capabilities and query tooling to both model and integrate associations of context information to referenceable sets of statements within the local scope of data exchanges, storage systems and data processing in Web ecosystems. These associations allow data to be asserted based on combined requirements for data and context, such as validating user age against the availability of associated signatures and usage agreements. Evaluation in Subsection 5.1 confirms that both collection-based (e.g., stdreif, tterm) and graph-based (e.g., bngraph) models meet the proposed requirements for referenceability, immutability, and recursive context expression. Analysis of query evaluation in Subsection 5.2 shows that SPARQL queries over these models can scale linearly with dataset size in systems like RDF4J, though performance depends on the graph store and query engine.

The use of SPARQL Construct to assert data based on combined requirements of data and context both for incoming data exchanges and evaluations on local data storage provides a robust mechanism for data integration in Web ecosystems without imposing additional tooling requirements. Existing models such as ODRL that do not formalize the interpretation of their target references can be reused to model context associations in the local scope.

In future work, we aim to build on these capabilities with a focus on supporting policy expressions that enforce constraints on requested claims in the form of a combined requirement for data and context modeled in RDF.

7. Supplemental Material Statement

The datasets, queries and benchmarks used are made available online at https://anonymous.4open.science/r/ISWC2025eval-5258/README.md

References

1. Berners-Lee, T.: The semantic web and challenges. https://www.w3.org/2003/Talks/01-sweb-tbl/ (2003).
2. Lange, C., Langkau, J., Bader, S.: The IDS Information Model: A Semantic Vocabulary for Sovereign Data Exchange. In: Otto, B., Hompel, M. ten, and Wrobel, S. (eds.) Designing Data Spaces : The Ecosystem Approach to Competitive Advantage. pp. 111–127. Springer International Publishing, Cham (2022). doi:10.1007/978-3-030-93975-5_7
3. Jung, C., Dörr, J.: Data Usage Control. In: Otto, B., Hompel, M. ten, and Wrobel, S. (eds.) Designing Data Spaces : The Ecosystem Approach to Competitive

Advantage. pp. 129–146. Springer International Publishing, Cham (2022). doi:10.1007/978-3-030-93975-5_8
4. Damásio, C.V., Analyti, A., Antoniou, G.: Provenance for SPARQL Queries. In: Cudré-Mauroux, P., Heflin, J., Sirin, E., Tudorache, T., Euzenat, J., Hauswirth, M., Parreira, J.X., Hendler, J., Schreiber, G., Bernstein, A., and Blomqvist, E. (eds.) The Semantic Web – ISWC 2012. pp. 625–640. Springer, Berlin, Heidelberg (2012). doi:10.1007/978-3-642-35176-1_39
5. Hartig, O.: Querying Trust in RDF Data with tSPARQL. In: Aroyo, L., Traverso, P., Ciravegna, F., Cimiano, P., Heath, T., Hyvönen, E., Mizoguchi, R., Oren, E., Sabou, M., and Simperl, E. (eds.) The Semantic Web: Research and Applications. pp. 5–20. Springer, Berlin, Heidelberg (2009). doi:10.1007/978-3-642-02121-3_5
6. Hartig, O., Thompson, B.: Foundations of an alternative approach to reification in RDF. arXiv preprint arXiv:1406.3399. (2014).
7. Flouris, G., Fundulaki, I., Pediaditis, P., Theoharis, Y., Christophides, V.: Coloring RDF Triples to Capture Provenance. In: Bernstein, A., Karger, D.R., Heath, T., Feigenbaum, L., Maynard, D., Motta, E., and Thirunarayan, K. (eds.) The Semantic Web - ISWC 2009. pp. 196–212. Springer, Berlin, Heidelberg (2009). doi:10.1007/978-3-642-04930-9_13
8. Carroll, J.J., Bizer, C., Hayes, P., Stickler, P.: Named graphs, provenance and trust. In: Proceedings of the 14th international conference on World Wide Web. pp. 613–622. Association for Computing Machinery, New York, NY, USA (2005). doi:10.1145/1060745.1060835
9. Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L.J., Coppens, F., Fernández, J.M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., others: Packaging research artefacts with RO-Crate. Data Science. 5, 97–138 (2022).
10. Kirrane, S., Mileo, A., Decker, S.: Access control and the resource description framework: A survey. Semantic Web. 8, 311–352 (2016).
11. Ianella, R.: Open digital rights language (ODRL). Open Content Licensing: Cultivating the Creative Commons. (2007).
12. Esteves, B., Pandit, H.J., Rodríguez-Doncel Víctor: ODRL profile for expressing consent through granular access control policies in solid. In: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 298–306. IEEE (2021).
13. Frey, J., Müller, K., Hellmann, S., Rahm, E., Vidal, M.-E.: Evaluation of metadata representations in RDF stores. Semantic Web. 10, 205–229 (2019).
14. Hogan, A., Arenas, M., Mallea, A., Polleres, A.: Everything you always wanted to know about blank nodes. Journal of Web Semantics. 27-28, 42–69 (2014). doi:10.1016/j.websem.2014.06.004
15. Dedecker, R., Colpaert, P., De Roo, J., Esteves, B.: Demonstrating a pragmatic solution to context associations in RDF using Blank Node Graphs. In: Proceedings

of the Extended Semantic Web Conference (2025).
16. Content Negotiation by Profile. https://www.w3.org/TR/dx-prof-conneg/

1. Thematic data spaces: https://digital-strategy.ec.europa.eu/en/policies/data-governance-act ↩
2. IDSA provenance tracking: https://docs.internationaldataspaces.org/ids-knowledgebase/open-dei-building-blocks-catalog/provenance_and_traceability ↩
3. IDSA use-cases: https://internationaldataspaces.org/wp-content/uploads/dlm_uploads/Use-Case-Brochure_2018.pdf ↩
4. Art. 15 GDPR: https://gdpr-info.eu/art-15-gdpr/ ↩
5. Art. 20 GDPR: https://gdpr-info.eu/art-20-gdpr/ ↩
6. Dublin core ontology: https://www.dublincore.org/specifications/dublin-core/dcmi-terms/ ↩
7. Provenance Ontology (PROV-O): https://www.w3.org/TR/prov-o/ ↩
8. Gaia-X usage ODRL: https://gaia-x.eu/gaia-x-and-the-policy-reasoning-engine/ ↩
9. IDSA usage ODRL: https://international-data-spaces-association.github.io/DataspaceConnector/Documentation/v5/UsageControl ↩
10. ODRL community group: https://www.w3.org/community/odrl/ ↩
11. ODRL use-cases: https://www.w3.org/community/odrl/wiki/Use_Cases ↩
12. W3C Verifiable Credentials” https://www.w3.org/TR/vc-overview/ ↩
13. ODRL profile for Verifiable Credentials: https://gitlab.com/gaia-x/lab/policy-reasoning/odrl-vc-profile ↩
14. RDF Dataset Canonicalization https://www.w3.org/TR/rdf-canon/ ↩ ↩[2]
15. Context Association specification: https://w3id.org/context-associations/specification ↩
16. SPARQL Construct creating graphs in Jena: https://jena.apache.org/documentation/query/construct-quad.html ↩
17. Evaluation codebase: https://anonymous.4open.science/r/ISWC2025eval-5258/ ↩